

Migration Test Strategy

Legacy on-prem system → modernized core banking platform

Owner: Keoni Kakugawa, Test Manager

Scope: Data migration validation for the core banking platform — customers, accounts, balances, and transactions moving from the legacy on-premises system to the modernized platform.

Objective

Provide evidence-based go/no-go assurance that every record migrated **completely, accurately, securely, and without performance regression** — and that any defect is caught in test, not in production.

Validation Categories

#	Category	What it proves	Method
1	Reconciliation	No records lost or duplicated	Source-vs-target row counts per table; signed delta
2	Referential integrity	Every relationship intact	Orphan detection via LEFT JOIN ... WHERE key IS NULL
3	Transformation / mapping	Converted fields translated correctly	Rule-based CASE mapping; catches wrong and invalid values
4	Security / PII	No sensitive data exposed	Plaintext-shape detection across all PII fields; access controls
5	Performance	No speed regression	EXPLAIN QUERY PLAN index-usage checks; p95 SLA thresholds

Test Approach

- **Source-to-target, automated, repeatable** — every check is codified and re-runs identically after each migration iteration; no manual eyeballing, which doesn't scale and misses defects.
- **Risk-based prioritization** — security/PII and performance are cutover blockers; a single exposed SSN or a dropped index halts go-live.
- **Root-cause over symptom** — one defect can surface across multiple checks (a dropped customer appears as both a count variance and an orphaned account); trace to the underlying cause.
- **Engineering partnership** — Test Engineers author and maintain validation queries; the Test Manager owns coverage, execution oversight, interpretation, and sign-off.
- **Prevention, not just detection** — recommend controls (constraints, enforced foreign keys, masking gates) so defects can't recur.

KPIs — Measuring Testing Effectiveness

KPI	Definition	Target
Reconciliation pass rate	% of tables/records that reconcile source-to-target with zero variance	100% before cutover
Defect leakage rate	% of defects found after the stage that should have caught them (escaped to later phases / production)	Trend to near-zero
Validation coverage	% of tables, fields, and categories covered by automated checks	100% of in-scope entities
Open cutover-blockers	Count of unresolved critical (security, data-loss, performance) defects	Must be 0 to go live

Team Structure

A **Test Manager + Test Engineer hybrid** model. The **Test Manager** owns strategy, coverage, risk prioritization, results interpretation, stakeholder reporting, and go/no-go sign-off — hands-on enough to read, review, and direct the validation queries. **Test Engineers** author and maintain the validation SQL, build the automated execution/reporting, and triage findings with the Test Manager. Deep technical authorship stays with engineering while accountability for quality and release readiness sits with a single owner.

Cutover Go / No-Go Criteria

Cutover is approved only when: reconciliation pass rate = 100%; zero orphaned/invalid records; all transformed fields validated; **no plaintext PII anywhere in the target**; hot-path queries meet p95 SLA with no regression vs. legacy baseline; and zero open cutover-blockers. Any failure is an automatic no-go.

Backed by a live validation suite (5 categories, executable checks) and supporting artifacts: PII access-control checklist and performance SLA / load-testing approach. keonikaku.github.io/sql-data-validation